DTOs (Data Transfer Objects)

Overview



What is DTOs?

Why do we need DTOs?

Where does AutoMapper fit in?

What is Clean Architecture?

- DTO stands for Data Transfer Object.
- A DTO is a simple class used to transfer data between the client and the Web API. It defines exactly which data the API accepts from the client and which data it returns.
- In your existing Product Web API, you have a Product entity in the DAL. We can introduce DTOs in the ServiceLayer so that clients no longer need to send or receive the entire entity.
- Think of it this way:
- Entity (Product) represents the data model used by your application and database.
- DTO (ProductReadDto, etc.) represents the data contract exposed by your API.
- AutoMapper converts objects between the entity and DTO classes.

Mapping Domain (Entity) with DTO's using AutoMapper (Product Example)

Domain Entity (DAL)

Product.cs (Entity)

```
public class Product
{
    public int ProductId { get; set; }
    public string? ProductName { get; set; }
    public string? ProductCategory { get; set; }
    public double ListPrice { get; set; }
    public bool IsDeleted { get; set; }
    public virtual ICollection<Order> Orders
        { get; set; } = new List<Order>();
}
```

Domain / Entity

- Represents database table (Product)
- Contains all fields (including internal fields)
- Used inside DAL / Service layer
- Not exposed directly to API clients

Entity to DTO (mapping)

DTO to Entity (mapping)

AutoMapper Mapping Profile

ProductMappingProfile.cs

```
using AutoMapper;
using DAL.Models;
using ServiceLayer.DTOs;

public class ProductMappingProfile : Profile
{
    public ProductMappingProfile()
    {
        // Entity -> Read DTO (for GET responses)
        CreateMap<Product, ProductReadDto>();

        // Create DTO -> Entity (for POST)
        CreateMap<ProductCreateDto, Product>();

        // Update DTO -> Entity (for PUT)
        CreateMap<ProductUpdateDto, Product>();
    }
}
```

AutoMapper

- Automatically maps properties with same names
- Converts between Entity and DTO objects
- Reduces manual mapping code

DTO Classes (ServiceLayer)

ProductReadDto (Used for HTTP GET responses)

```
public class ProductReadDto
{
    public int ProductId { get; set; }
    public string? ProductName { get; set; }
    public string? ProductCategory { get; set; }
    public double ListPrice { get; set; }
}
```

Returned to client (GET)

ProductCreateDto (Used for HTTP POST)

```
public class ProductCreateDto
{
    public int ProductId { get; set; }
    public string? ProductName { get; set; }
    public string? ProductCategory { get; set; }
    public double ListPrice { get; set; }
}
```

Sent by client (POST)

ProductUpdateDto (Used for HTTP PUT)

```
public class ProductUpdateDto
{
    public string? ProductName { get; set; }
    public string? ProductCategory { get; set; }
    public double ListPrice { get; set; }
}
```

Sent by client (PUT)

How Mapping Works in Each API Operation

1. HTTP GET (Entity → DTO)

Product (Entity) → _mapper.Map<ProductReadDto>(entity) → ProductReadDto (DTO)

Domain Entity (from DB)

```
{
  "productId": 101,
  "productName": "Laptop",
  "productCategory": "Electronics",
  "listPrice": 55000,
  "isDeleted": false,
  "orders": [....]
}
```

API Response (DTO)

```
{
  "productId": 101,
  "productName": "Laptop",
  "productCategory":
   "Electronics",
  "listPrice": 55000
}
```

2. HTTP POST (DTO → Entity)

ProductCreateDto (DTO) → _mapper.Map<Product>(createDto) → Product (Entity)

Request Body (DTO)

```
{
  "productId": 102,
  "productName": "Mouse",
  "productCategory":
   "Accessories",
  "listPrice": 800
}
```

Mapped Entity (to save)

```
{
  "productId": 102,
  "productName": "Mouse",
  "productCategory":
   "Accessories",
  "listPrice": 800,
  "isDeleted": false,
  "orders": []
}
```

3. HTTP PUT (DTO → Existing Entity)

ProductUpdateDto (DTO) → _mapper.Map(updateDto, existingEntity) → Existing Product (Entity)

Request Body (DTO)

```
{
  "productName": "Updated
   Laptop",
  "productCategory":
   "Electronics",
  "listPrice": 60000
}
```

Existing Entity after mapping

```
{
  "productId": 101,
  "productName": "Updated
    Laptop",
  "productCategory":
   "Electronics",
  "listPrice": 60000,
  "isDeleted": false,
  "orders": [....]
}
```

4. HTTP DELETE (No mapping required)

ProductId (from URL) → Service Layer (RemoveProduct(id))

DELETE only requires the product ID.
No DTO is needed for request.
You can optionally return a response DTO.

Optional Response DTO

```
{
  "productId": 101,
  "message": "Product deleted successfully.",
}
```

Why do we need DTOs?

1. Security

A client should not be able to set internal properties such as IsDeleted simply by including them in a POST request. A dedicated create DTO accepts only the fields intended for client input.

2. Control over API responses

You can return only ProductId, ProductName, ProductCategory, and ListPrice, instead of exposing every entity property.

3. Input validation

ProductCreateDto can use attributes such as [Required], [StringLength], and [Range] to validate incoming requests.

4. Separation of concerns

Your database entity can change without necessarily changing the JSON contract used by API clients. DTOs make it easier to evolve the database model and the API independently.

Where does AutoMapper fit in?

DTOs and AutoMapper are related, but they are not the same thing.

- DTOs define the shape of the data.
- AutoMapper performs the object-to-object conversion.
- Your existing ProductService and DAL continue to perform the data-access operations.

GET: Entity → DTO

```
_mapper.Map<ProductReadDto>(product)
```

Return only the approved fields to the client.

POST: DTO → Entity

```
_mapper.Map<Product>(newProductDto)
```

Convert the validated input into an entity for saving.

PUT: DTO → Existing Entity

```
_mapper.Map(updatedProductDto, existingProduct)
```